

Assignment-1: Benchkit

Milan Lagae

University Name: Vrije Universiteit Brussel

Faculty: Sciences and Bioengineering Sciences

Course: Performance Analysis & Evaluation

May 10, 2026

Contents

1. Intro	3
2. Context	3
3. Bench	4
3.1. Fetch	4
3.2. Build	4
3.3. Run	4
3.4. Collect	4
4. Command wrapper	4
5. Analysis	5
5.1. Platform	5
5.2. Base	5
5.3. Wrapper	6
Bibliography	8

1. Intro

The benchmark chosen for porting to the updated Benchkit V2 campaign API is the `cuda_samples_matmul`. To start, context of the chosen benchmark will be discussed in Section 2. Modifications made will be discussed in Section 3, with use of a command wrapper in Section 4. Discussion of benchmark results will be done in Section 5.

2. Context

It is a matrix multiplication microbenchmark for testing the performance of performing matrix multiplication on a GPU's. It is tailored to test the performance of GPU's.

Specified in the benchmark variables are the dimensions of the matrix that are to multiplied with each other.

3. Bench

This section will briefly discuss the updated benchmark implementation;

3.1. Fetch

The fetch stage of the benchmark has been updated with cloning the completely example repository list in the original benchmarks documentation.

3.2. Build

The build stage is specially tailored to compiling and building the `matrixMul.cu` kernel at the moment. The `cmake` & `make` commands are sequentially executed for compiling said example.

3.3. Run

This stage executes a single program execution, existing bench code has been reused.

3.4. Collect

The existing collect stage has been copied and no modifications has been made to this stage code.

4. Command wrapper

The chosen wrapper for the updated benchmark is the `ncu2.py` wrapper, which is the cli version of NVIDIA Nsight Compute¹.

Using the GPU's hardware performance counters, more information from the execute kernel can be gained.

¹<https://developer.nvidia.com/nsight-compute>^o

5. Analysis

This section will discuss analyzing the results of executing the updated benchmark for the Cuda matrix multiplication example.

5.1. Platform

Due to platform availability the benchmarks were executed on a Ubuntu 22.04 WSL image on Windows 11. More information can be found in Table 1.

PART	VALUE
CPU	Ryzen 9 5950X
GPU	RTX 3070
Driver Version	595.97
RAM	64GB (3200 Mhz)
OS	Windows 11 - Version 10.0.22631 Build 22631
WSL Version	2
WSL Distro	24.04

Table 1: Desktop Specifications

5.2. Base

Included in the output of the base cuda kernel performance metrics are the following values:

- ma_width: Matrix A size.
- ma_height: Matrix A size.
- mb_width: Matrix B size.
- ma_width: Matrix A size.
- dim_a: Matrix A dimensions.
- dim_b: Matrix B dimensions.
- dim_a_x_dim_b: Matrix A & B dimensions combined.
- GFlops/s: Compute throughput per second.
- Time ms: Execution time.
- Ops: Total number of operations executed.
- Workgroup Size: Indicates the number of threads in a work group.

5.2.1. Graphs

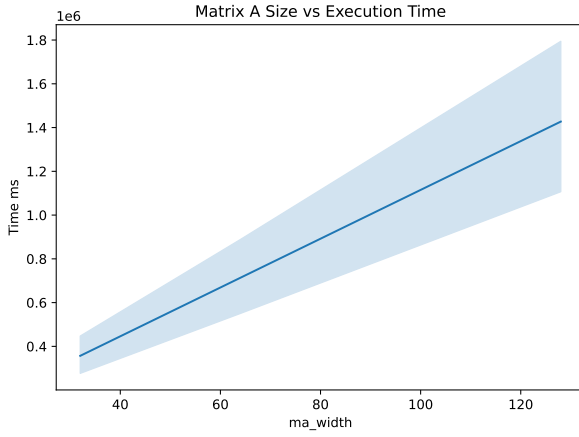


Figure 1: Old - Size vs Execution Time

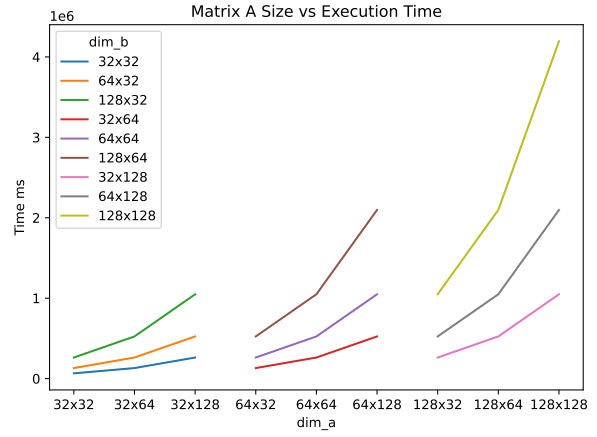


Figure 2: New - Size vs Execution Time

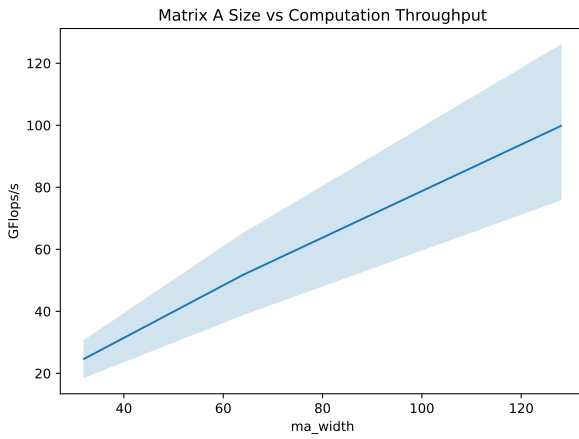


Figure 3: Old - Size vs Throughput

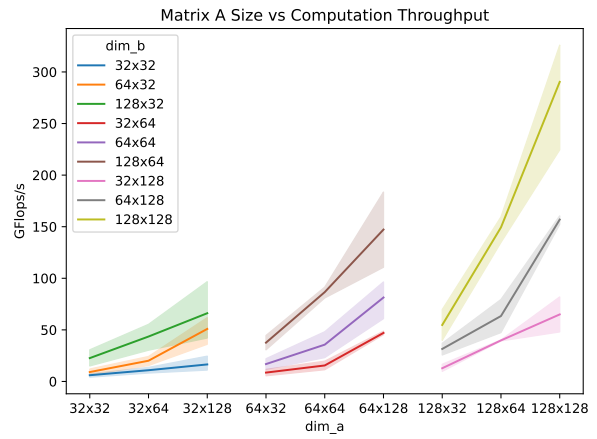


Figure 4: New - Size vs Throughput

5.3. Wrapper

The default metric included in the wrapper for analysis is: `smsp__sass_l1tex_tags_mem_global`. Using the command: `ncu --query-metrics --metrics smsp__sass_l1tex_tags_mem_global` the description of this metric can be queried. The description reads as follows: “# of L1 cache tag lookups generated by global memory instructions”

5.3.1. Graphs

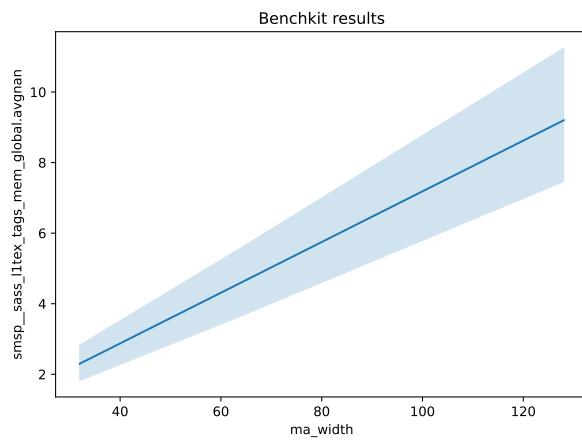


Figure 5: Old - Size vs NCU Metric

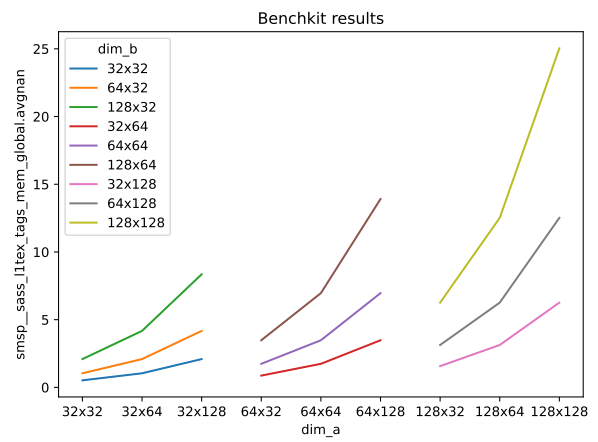


Figure 6: New - Size vs NCU Metric

Bibliography